EBU6230 – Image and Video Processing – 2022/23

Coursework report and exercises

Name: ____Tangxun Wu_____

Username: _____jp2020213248___200978947_____

Exercise 1 (a)

Reading/writing PGM/PPM images: The first step towards image and video processing is reading images from a file and write them to a file. There exist different standards that store the information in different formats; so before opening an image, knowledge of the standard is necessary.

Two widely used image formats are PPM and PGM. The PGM format is a greyscale file format designed to be easy to manipulate. A PGM image represents a greyscale graphic image. For most purposes, a PGM image can just be thought of as an array of integers. The name "PGM" is the acronym of "Portable Grey Map." The name "PPM" is the acronym for "Portable Pixel Map." Images in this format (or a precursor of it) were once also called "portable pixmaps." It is a highly redundant format, and contains a lot of information that the Human Visual System (HVS) cannot even discern. However, as for PGM, PPM is very easy to write and analyse.

The goal of the first part of today's lab is to become comfortable with these two formats. You will implement functions to read and to write PPM and PGM images. The final demonstration of the implemented software will be done using the well-known test images: LENA, BABOON, PEPPERS, etc. You can find PPM and PGM versions of these images in the EBU6230 QMplus pages. The writing function must add as a comment in the header: "image created by *your_name*".

Include in your submission the file resulting from reading the images provided and writing them back in their original format.

Summarize in 5 points the operations necessary to read a PGM/PPM image:

1. First, open and read the file (including images and header), then distinguish the image type (PGM:P2 and P5 or PPM:P3 and P6).

2. Second, when reading the images in file, ignore the comment lines starting with # in header.

3. Third, read the information of header, including width, height, format, and maximum level of image.

4. Fourth, read the images with correct format: ASCII format is used for P2 and P3 images, and binary format is used for P5 and P6 images, which are required to transform into Unit8 format.

5. Fifth, read and record the image data into matrix, then close the file.

Summarize in 5 points the operations necessary to write a PGM/PPM image:

1. First, after opening the image file through write mode, define the image type (PGM:P2 and P5 or PPM:P3 and P6) in the beginning.

2. Second, when writing the images into file, write the comment lines starting with # in header. And each line should be ended by \r\n in windows system (change to be \n in Unix system).

3. Third, write the information of header, including width, height, format, and maximum level of image.

4. Fourth, write the images with correct format: write P5 and P6 file in utf8 format as these images are encoded by binary, write P2 and P3 file in unit8 format as these images are encoded by ASCII.

5. Fifth, write the image data into matrix while using correct format, then close the file.

What is the difference between the identifiers P3 and P6?

1. the P3 identifier means the PPM colour image is stored in ASCII, while the P6 identifier means the PGM image is stored in binary format.

2. the P6 identifier means the image file can only be used to show single byte colours (one byte per colour component), while image files with P3 identifier don't have this limitation, the value of each pixel in P3 PPM image can range from 0 to maximum value given in the header.

3. For the lines: comments in P6 image can be only written before the last field of the header and one byte after the last header field. The lines in P3 image couldn't be longer than 70 characters.

4. image with P6 identifier is smaller than image with P3 identifier, which is faster to read.

Exercise 1 (b)

Format conversions: in this part of the lab, the images will be converted from colour to grey scale; in other words a PPM image will be converted to the PGM format. You will implement a function called "BUPT_format_converter" which transforms images from colour to grey-scale using the following YUV conversion:

$Y = 0.257 * R + 0.504 * G + 0.098 * B + 16$

$U = -0.148 * R - 0.291 * G + 0.439 * B + 128$

V = 0.439 * R - 0.368 * G - 0.071 * B + 128

Note swap of 2nd and 3rd rows, and sign-change on coefficient 0.368

What component represents the luminance, i.e. the grey-levels, of an image?

"Y" component represents the luminance of an image.

Use thee boxes to display the results for the colour to grey-scale conversion.

Lena colour (RGB)



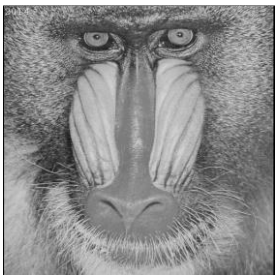
Lena grey



Baboon colour (RGB)



Baboon grey



Is the transformation between the two colour-spaces linear? Explain your answer.

In my opinion, the transformation between the two colour-spaces is linear.

Reason:

From the transformation formula given above:

$$Y = 0.257 * R + 0.504 * G + 0.098 * B + 16$$

$$U = -0.148 * R - 0.291 * G + 0.439 * B + 128$$

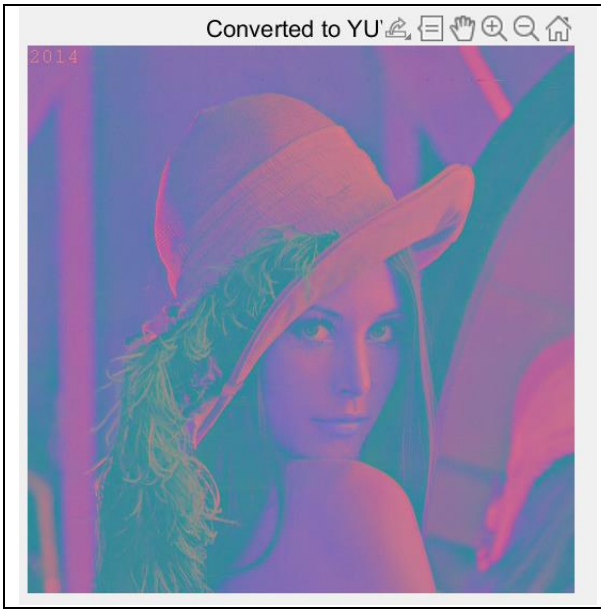
$$V = 0.439 * R - 0.368 * G - 0.071 * B + 128$$

we can see that the conversion shows the linear formula: “y = ax + b”.

e.g. For Y, y = 0.257 * R + 0.504 * G + 0.098 * B + 16, where a = [0.257, 0.504, 0.098], x = [R,G,B]’, b = 16.

Also, they both use three bytes to represent colours, which are converted by a fixed linear mapping between them. Therefore, the transformation is linear.

Display in the box the Lena image converted to YUV 3 channels format.



Are the colours of the previous picture distorted? If yes why?

Yes, I think the colours are distorted.

First, in Matlab calculations the Y, U, V components are using uint8(round()) to get integer result, and the function utilize the 3 integer components to create the YUV picture. Therefore, the picture will be distorted.

Second, in Matlab I use imshow() function to present the picture after transformation, and this function still treats the picture after transformation as a RGB picture. Therefore, we cannot avoid distortion during the process.

Based on the formula for the RGB to YUV conversion, derive the formula for the YUV to RGB conversion.

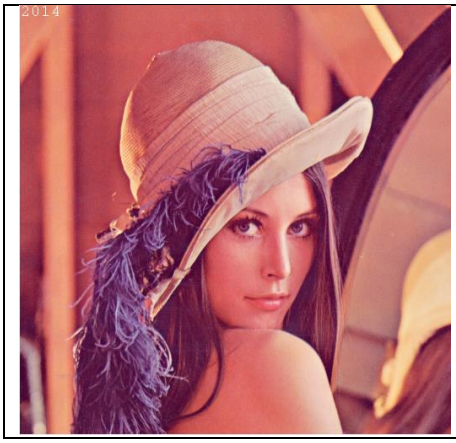
Based on RGB to YUV Conversion, I get the formula for RGB:

R = 1.164(Y - 16) + 1.596(V - 128) = 1.164*Y + 1.596*V - 222.912

G = 1.164(Y - 16) - 0.813(V - 128) - 0.391(U – 128) = 1.164*Y - 0.391*U - 0.813*V + 135.488

B = 1.164(Y - 16) + 2.018(U – 128) = 1.164*Y + 2.018*U - 276.928

Use the formula you derived at the previous step to convert the YUV image back to the original RGB format. Display the result in the box.



Exercise 1 (c)

Sub-sampling: The HVS is incapable of perceiving certain details in an image. Therefore high compression ratios can be achieved by exploiting the characteristics of the HVS, thus discarding what has a low visual relevance. However, this process can introduce distortions due to the compression. A simple way to exploit the characteristics of the HVS to give compression is to sub-sample an image. A drawback of this approach is that it is possible to incur the well-known problems of a discrete representation, such as aliasing. This part of the lab covers some simple sub-sampling operations.

Implement a function that sub-samples grey level images by a factor n, with n a multiple of 2. The function should be able to sub-sample independently in the horizontal and in the vertical direction or in both directions at the same time.

Display the results of sub-sampling the image Lena using the following factors: 2 horizontal, 2 vertical, 2 vertical and 8 horizontal, 4 vertical and 4 horizontal. Include the files of the results in the submission.

Box for the 4 images

Loading the “Lena512_Binary2014.pgm” image to Matlab:

2 horizontal:



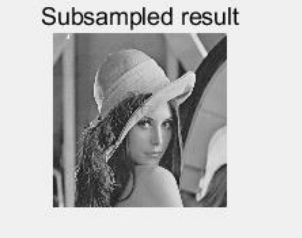
2 vertical:



2 vertical and 8 horizontal:



4 vertical and 4 horizontal:



Describe, using your own words, the aliasing problem and how to avoid it, as applied to signal processing

The aliasing problem is caused by low sampling rate. When doing sub-sampling, which means too many pixels are removed according to a given pattern in the image, aliasing problem will occur, and the spatial resolution also becomes pretty low.

Besides, by Nyquist's theorem, when the sampling frequency is less than twice of highest frequency of the image, the process of sampling will lead to aliasing problem in image.

To avoid it: in practice, we should ensure that the sampling frequency is at least twice of highest frequency of the image, so that the number of sampling points in spatial domain is reassured. Also, after doing sampling, there can be a low-pass filter to remove the components whose frequency will cause aliasing.

Given a scene sampled by a ccd sensor with minimum horizontal sampling frequency $10cm^{-1}$, what is the maximum horizontal frequency in the image that can be correctly represented?

According to Nyquist's theorem talking above, the image can be correctly represented when the sampling frequency is greater than twice of highest frequency of the image.

Therefore, the maximum horizontal frequency is 5 cm^{-1}, which is 1/2 of the minimum horizontal sampling frequency.

If you sub-sample an image, why do you have more problems from aliasing?

Because when doing sub-sampling, pixels are removed according to a given pattern. In this case, information in rows or columns are thrown away and spatial resolution are decreased. Not only there will be overlapping of information after sampling, but also some parts of information are missing after sub-sampling.

Therefore, there are more problems from aliasing.

Paste below a clear example of artefacts generated by aliasing. For this task you can use your own choice of image. Use the box below for the image and comments.

Original ".pgm" image:



Subsampled by 4 vertical rate and 4 horizontal rate: (I zoom the subsampled image already)



Subsampled by 8 vertical rate and 8 horizontal rate: (I zoom the subsampled image already)



After sub-sampling, I adjust the image in same size because the image is too small.

It's clear to see that the images are blocky and of low quality.

Exercise 2 (a)

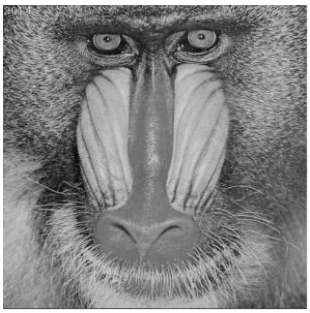
Quantize: Quantization is the process of approximating the continuous values in the image data with a finite set of discrete values. The input of a quantizer is the original data and the output is one among the finite number of levels. This process is an approximation, and a good quantizer is one which represents the original signal with minimum loss (quantization error). In this lab, you will work with a scalar uniform quantizer applied to grey-scale images.

Implement a function that uniformly quantizes grey level images. The function will allow the reduction of the number of grey level values by a given factor n (a power of 2). *Note*. To visualize the image, you need to re-map it in the 8-bit-per-pixel representation. Show the results in the boxes below.

Lena, quantization factor 2



Baboon, quantization factor 8



Peppers, quantization factor 32



Peppers, quantization factor 128



Is quantization a reversible process? Can you recover what you discarded? Briefly explain.

No. It is not a reversible process. The purpose of quantization is to map a set of values to only a close value, so that transforming a group of continuous values into discrete values. During the process, some colour details are lost. Therefore, we cannot recover what we discard after quantisation.

Write the results back to PGM/PPM files using the function you created. Make sure that your writing function allocates the correct number of bits per pixel. What is the size of the files compared with the original? Given the results, what is a typical application field for quantization? Include in your submission the output files and comment on the results.

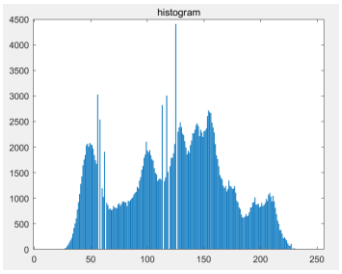
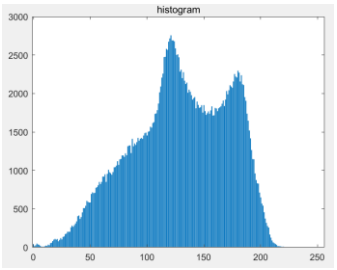
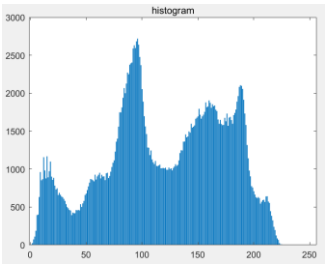
The size of files after quantization becomes smaller, that is 1/N of the original files, where N is the quantisation factor.

Because the quantization process can decrease the size of files, it can be applied to do data compression. It can obtain “almost lossless” file with a small precision loss cost.

Exercise 2 (b)

Histograms: This part of the lab is dedicated to image processing using histograms. A histogram is a statistical representation of the data within an image. The histogram can be represented as a plot of the frequency of occurrence of each grey level. This representation shows the distribution of the image data values. By manipulating a histogram, it is possible to improve the contrast in an image and the overall brightness or to segment different areas of the image by applying one or more thresholds to the histogram itself.

Implement a function to output the histogram values of a given grey level image. Display in the boxes the resulting histograms.

Lena	Baboon	Peppers
		

If you normalize the values of the histogram so that they sum to 1, what does the value of a bin represent?

After normalizing the values of the histogram,

the value of a bin represents the proportion of each different grey level in the total number of grey levels.

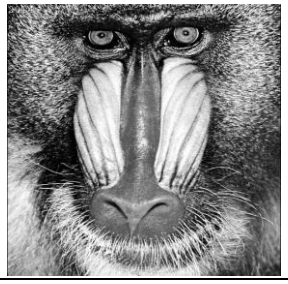
It shows a probability distribution of grey levels through the entire image.

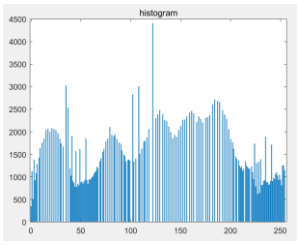
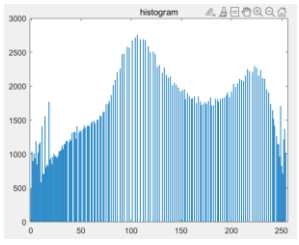
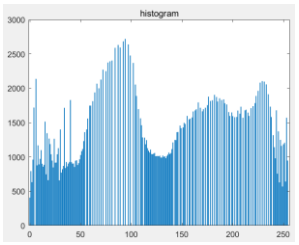
Exercise 2 (c)

Equalize: Equalization is one of the possible image processing algorithms implemented using histograms. Histogram equalization allows a user to enhance the contrast of images. Histogram equalization employs a monotonic, non-linear mapping which re-assigns the intensity values of pixels in the input image such that the output image contains a uniform distribution of intensities (i.e. a flat histogram).

Implement a function that equalizes grey-scale images based on their histogram. The input is a given grey level image; the output is the derived image with uniform intensity distribution.

Display in the boxes the equalized images and their histograms.

Lena equalized	Baboon equalized	Peppers equalized
		

Lena histogram after equalization	Baboon equalized after equalization	Peppers equalized after equalization
		

Are the distributions really uniform? Explain your results.

No. The distribution is not uniform.

In contrast of original histogram and histogram after equalization, it's clear that the histogram after equalization haven't achieved evenly and uniform distribution. Because it just redistributes the frequency that each grey level occurs but doesn't change the number of grey levels (the pixel number).

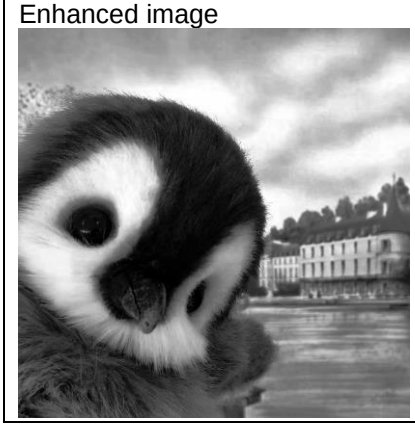
Therefore, it cannot strictly let distributions become uniform, but by adjusting the frequency distribution, it can let the change of greyscale in picture becomes less sharp and smoother.

Show an example of the successful application of histogram equalization to image enhancement. You can use an appropriate image of your choice

Original image



Enhanced image



Comment on the results of the previous step.

After the step, the lightness and contrast of the image I chose are enhanced and improved.

For example, the cloud in the sky becomes lighter, while the noise of penguin becomes darker.

Besides, through increasing the contrast, we can see the shape and contour of the objects in the image more clearly.

Exercise 2 (d)

Histogram modelling: Histogram modelling techniques are effective tools for modifying the dynamic range and contrast of an image. Unlike contrast stretching, histogram modelling operators may employ non-linear and non-monotonic transfer functions to map between pixel intensity values in the input and output images. In the first part of this lab you will model the histogram of a grey-scale image.

Implement a point to point operation that maps the input grey level image into an output image which has a predefined frequency distribution. The algorithm is not given explicitly in the lecture slides, you are supposed to derive it. Use as input histogram the histogram of an image A and model the histogram of another image B according to the input.

(A = Lena) (B=Peppers)

Image A



Histo of A

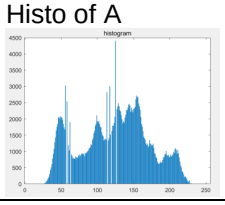
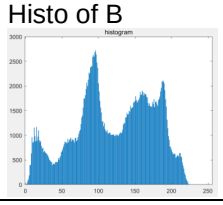


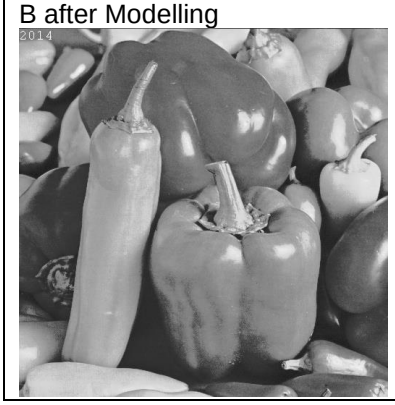
Image B



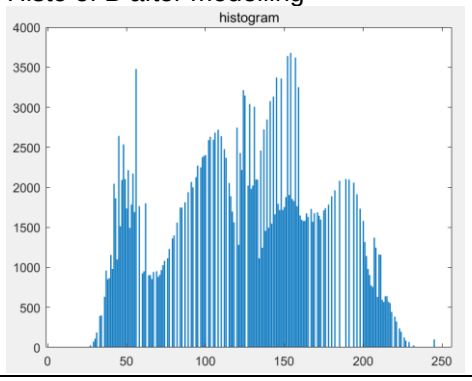
Histo of B



B after Modelling

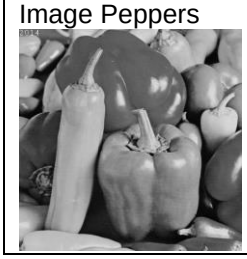


Histo of B after modelling

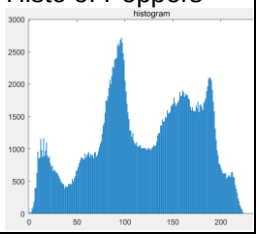


Use as input histogram an approximation of the exponential distribution.

Image Peppers



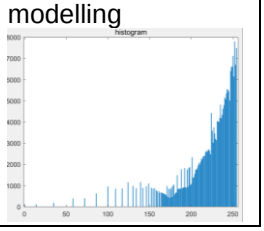
Histo of Peppers



Peppers after modelling



Histo after modelling



Write in the box the formulation of your algorithm.

```
%create the exponential distribution
for i=-255:0
    e(i+256)=1.6^(i/16);
end

%create the cumulative probability distribution
cd1=[];
for i=1:256
    cd1 = [cd1, e(i)];
end

%normalize the pixel probability
histogram=imhist(in2);
pt=histogram/(h2*w2);

%create the cumulative probability distribution
cd2=[];
for i=1:256
    cd2=[cd2, sum(pt(1:i))];
end
```

Exercise 3 (a)

Negatives: We are used to the negative of an image in analogue image processing. It is possible to generate a negative from a digital image too. In the last part of today's lab you will solve a simple exercise on negative images.

Write a function that inverts the grey level of a PGM image (i.e. it creates the negative of the image).

Negative of Lena



Negative of Peppers



Negative of Baboon



Perform the same task with PPM images and comment on the results.

Negative of Lena



Negative of Peppers



Negative of Baboon



Your comments:

The negative image can be obtained through subtracting the input image matrices from 255, that is: image = 255 – input_matrix.

For colour image, the calculation is implemented through 3 RGB channels.

Exercise 3 (b)

Rotation and translation. Image processing toolboxes allow a user to rotate, translate and skew images. These are very useful operations for image composition, for example. The first exercise will cover the implementation of two such transformations.

Write a function *BUPT_transform* that takes as input an image *I*, rotates it with an angle θ_1 and skews it with a second angle, θ_2.

Write the matrix formulation for image rotation (define all variables).

When rotating counterclockwise:

writting in 2D format:

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

writing in 3D format:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

x, y are the original coordinates, while x' and y' are coordinates after doing rotation. Θ is the rotating angle.

In this case, I suppose rotate Θ degrees counterclockwise. If rotating clockwise, the angle should be changed to -Θ.

Write the matrix formulation for image skewing (define all variables).

writting in 2D format:

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} 1 & \frac{1}{\tan\theta} \\ 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

writing in 3D format:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & \frac{1}{\tan\theta} & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

x, y are the original coordinates, while x' and y' are coordinates after doing skew. Θ is the skewing angle.

Create and paste below a PGM image containing your name written in Arial font, point 72, uppercase letters.



TANGXUN WU

Rotate the image you created by 30, 60 120 and -50 degrees clockwise and display the results below.

30



60



120



-50



Skew the same image by 10, 40 and 60 degrees and display the results below.

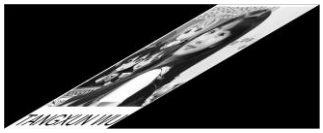
10



40



60



During the development process have you experienced the problem of regular patterns of black pixels in the image? If so, explain how you solved the problem. Otherwise imagine what could have generated these artefacts and how you would have worked around them.

I have experienced the problem of regular patterns of black pixels in the image.

First, after transformation, some output pixels can't map with the original input pixels. Second, during the transformation we use round() function, which may decrease information of pixels. Therefore, the regular patterns of black pixels appear as the background of image are black.

To deal with it, I choose to use the inverse of warp W by doing interpolation: loop through all destination pixels, then find the corresponding source pixels. After that, by interpolating in source image around source position, find the final value for the destination pixels.

Rotate the image by 20 degrees clockwise and then skew the result by 50 degrees. Display the result in 'a'.

Skew the image by 50 degrees and then rotate the result by 20 degrees clockwise. Display the result in 'b'.

a



b



Analyse the results when you change the order of the two operators i.e. R(S(*I*)) and S(R(*I*)), where R is the rotation and S is the skew. Are the results of (a) and (b) the same? Why?

The results are not the same. S(R(I)) causes the image flatter than R(S(I)).
Because the operation is done through transformation matrix as inputs. When multiplying matrixes, A*B is not always equal to B*A unless A and B are diagonal matrixes.

Therefore, the different operation procedures will cause different image results.

Exercise 4 (a)

Noise and PSNR: This part of the lab introduces error metrics for image quality evaluation. Two common metrics are the Mean Square Error (MSE) and the Peak Signal to Noise Ratio (PSNR). The MSE is the cumulative squared error between the processed image and the original image. The PSNR makes use of the MSE. The smaller the MSE, the smaller the error is. The larger the PSNR, the smaller the error is. You will add different amounts of random noise to the test images and measure their MSE and PSNR.

Write the formulas of MSE and PSNR.

$$MSE = \frac{\sum[I(x,y) - I'(x,y)]^2}{N/M}$$

$$PSNR = 20\log_{10}\left(\frac{255}{\sqrt{MSE}}\right)$$

Can the PSNR return a negative value? Explain your answer.

The PSNR cannot return a negative value.

After the processing, usually the output processed image will be disparate from original image to some extent. From the formula written above, when MSE > 255^2, PSNR will be negative. However, the difference between the original image and the processed image cannot cause MSE become bigger than 255^2.

Only when the processed image is totally different from the original image, e.g., the original image is white, while the processed image is black, can the MSE become 255^2 and will the PSNR become negative. But this cannot be realized in this case.

Create a function that can add (i) salt and pepper noise and (ii) Gaussian noise to a PGM image and compute PSNR and MSE. Show the results in the box and write under the box the values of MSE and PSNR comparing the original with the corrupted one.

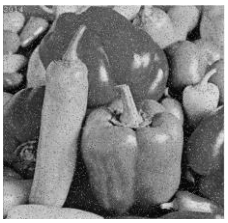
Lena salt and pepper noise

MSE = 6.40 PSNR = 40.07



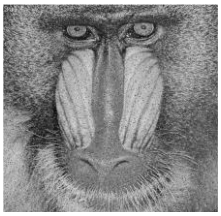
Peppers salt and pepper noise

MSE = 6.35 PSNR = 40.10



Baboon salt and pepper noise

MSE = 6.28 PSNR = 40.15



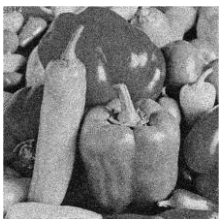
Lena Gaussian noise σ =1% of the range.

MSE = 86.56 PSNR = 28.76



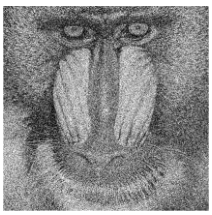
Peppers Gaussian noise σ =2%

MSE = 97.78 PSNR = 28.23



Baboon Gaussian noise σ =7%

MSE = 111.69 PSNR = 27.65



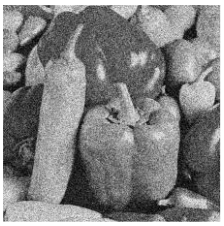
Lena Gaussian noise σ =5% of the range.

MSE = 108.67 PSNR = 27.77



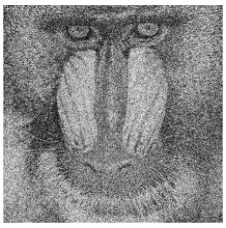
Peppers Gaussian noise σ =5%

MSE = 108.71 PSNR = 27.77



Baboon Gaussian noise σ =10%

MSE = 114.03 PSNR = 27.56



Exercise 4(b)

Up-sampling: Scaling-up an image (up-sampling) requires the filling of the new positions given the original pixels. This filling can be obtained by interpolation. Different interpolation techniques can be used. The choice depends on the quality we want to achieve and on the computation resources we have available. The nearest-neighbour interpolation is the simplest and fastest technique, but it is also a technique achieving low quality results. Bilinear interpolation is computationally more intensive, but it achieves higher quality results.

Implement the function *BUPT_up* that increases the resolution of images by a given factor (also a non-integer one). The up-sampling should be achieved using the nearest neighbour as well as the bilinear interpolation. The function will be able to up-sample independently in the horizontal and in the vertical direction or in both directions simultaneously.

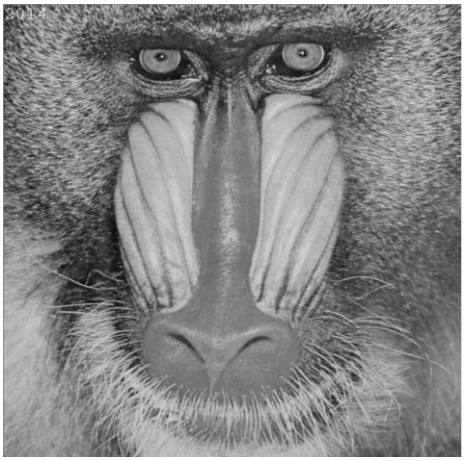
Up-sample the image Lena using nearest neighbour interpolation. Display a blow-up of the image Lena obtained by up-sampling the original image with factor 4.5. The image should clearly show the type of artefact obtained using the nearest neighbour interpolation. Use the box below to display the image and discuss the results.



Your comments:

Nearest neighbour interpolation is completed by generating pixels through copying the nearest available pixels. Therefore, there are some artifacts (sawtooth) and errors at unknown locations and the image details are blurred to some extent, because their values are estimated and calculated.

Up-sample the image Baboon using bilinear interpolation. Paste below a zoomed portion of the image Baboon obtained by up-sampling the original image with a factor 3.6. Discuss the artefacts obtained using bilinear interpolation.



Your comments:

The procedure of bilinear interpolation is to interpolate linear on each horizontal edge, then using obtained results to interpolate in vertical direction. It uses 4 known pixels on the regular grid to calculate the inside pixel.

Looking at the processed picture, compared to the picture completed by nearest neighbour interpolation, this picture is less blurred and blocky. Also, in the boundary where colours change, the colour transformation is smoother and silkier.

Compare the nearest neighbour technique and the bilinear technique in terms of speed and accuracy. Which technique is faster? Why? What artefacts are more visually disruptive?

1. nearest neighbour interpolation is faster, because it just collects one nearest value to estimate unknown data, while bilinear interpolation uses 4 known pixels and has more complex algorithm.

2. bilinear interpolation has better accuracy, while nearest neighbour interpolation makes artefacts become disruptive and discontinuous. For bilinear interpolation, the intensity value and image details can be well maintained.

Exercise 5(a)

Low pass filtering: Image filtering generates a processed image as a result of certain operations on the pixels of the original image. Each pixel in the output image is computed as a function of one or several pixels in the original image, usually located near the output pixel. The procedure is usually implemented by convolving a kernel with desired properties with the pixels of the input image. If the kernel is a Gaussian kernel, then the behaviour of the filter depends on the variance of the Gaussian.

Write a function BUPT_lowpass that convolves an image with a Gaussian kernel.

(i) Write the formula of the kernel you used.

(ii) The 2D Gaussian kernel is separable: write the two separate equations for the rows and the columns, and discuss the advantages of using separable filters.

(iii) What is the relationship between σ and the cut-off frequency of the filter?

(iv) Given σ, what criterion should be used to choose the size of the kernel? Why?

Your comments:

(i)

Gaussian Filter

$$G_\sigma(k,l) = \frac{1}{2\sigma^2\pi} e^{-\frac{k^2+l^2}{2\sigma^2}}$$

(ii)

$$G_\sigma(k,l) = G_\sigma(k)\, G_\sigma(l)$$

$$G_\sigma(k) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{k^2}{2\sigma^2}} \qquad G_\sigma(l) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{l^2}{2\sigma^2}}$$

the advantages of using separable filters: we can first filter the rows of input image, and then filter the columns of result. This can improve the speed of calculation, letting process be faster than using 2D kernel.

(iii) relationship between σ and the cut-off frequency of the filter:

When σ is larger, the Gaussian waveform will be more dispersed and distributed from the cut-off frequency of the filter. When σ is smaller, the Gaussian waveform will be more concentrated in the cut-off frequency of the filter.

Besides, larger σ results in more filtering, more information being lost in the output image, more blurring, and more computational complexity.

(iv) choose the size of the kernel: as is known to all, in the interval ($\mu - 3\sigma$, $\mu + 3\sigma$) the area is 99.7% of the total area under the Gaussian curve. Therefore, we can determine the kernel radius to be 3σ and size of kernel to be 6σ+1 (need an anchor point).

Add Gaussian noise to the image Lena with noise power 50 dBm, then convolve the noisy image with Gaussian kernels with σ = 0.5, 1, 2, 4, 7, 10, respectively. Paste below the resulting images. Comment the results obtained with increasing values of σ.

σ=0.5



σ=1



σ=2



σ=4



σ=7



σ=10



Your comments:

When the σ becomes larger, the impacts of noise brought to image are decreasing.

However, the image become more blurred, and we can't get image details in this case.

Implement a rectangular-shaped filter (*BUPT_rect*). Filter the noisy image Lena with a 5-by-5 and with a 7-by-7 kernel. Paste below the resulting images. Compare these results with those obtained with the Gaussian filter.

5-by-5 pixel



7-by-7 pixel



Your comments:

The image produced by 5-by-5 pixel is clearer than image produced by 7-by-7 pixel, while image produced by 7-by-7 pixel reduces more noise and become smoother than image produced by 5-by-5 pixel.

Besides, compared images by Gaussian filter and rectangular filter, image by Gaussian filter has more natural effect and less degree of blur after smoothing. Edge information is saved as much as possible in Gaussian filter.

Exercise 5(b)

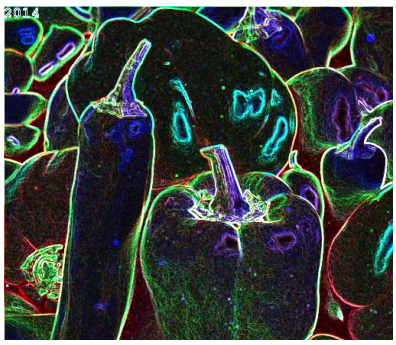
Edge Detection: Edge detection is the process of identifying and locating discontinuities in an image. The discontinuities are sharp changes in pixel intensity which characterise object boundaries. Classical edge detectors convolve the image with a 2-D kernel designed to be sensitive to large gradient amplitudes. There exist a large number of edge detectors, each designed to be sensitive to certain types of edges.

Implement the Sobel, Roberts and Prewitt filters for grey level and colour images. In the case of colour images, you can apply separate filtering of each of the three RGB components. Paste below the images representing the absolute value of the gradient for the three filters. Comment on how you dealt with the borders.

Sobel Lena (grey)



Sobel Peppers (colour)



Roberts Lena (grey)



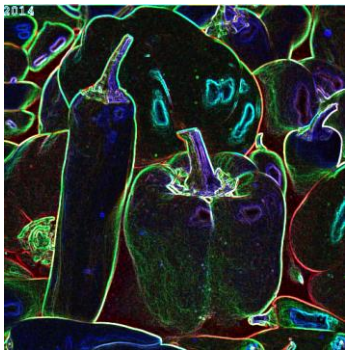
Roberts Peppers (colour)



Prewitt Lena (grey)



Prewitt Peppers (colour)



Your comments:

Among three filters, Sobel filter can maintain best quality of original image: it can display almost complete and clear edges in original image. Also, the image after Sobel filter has a high contrast, and the edges are located correctly.

Roberts filter works similar as Sobel filter, and it can still has a good influence on keeping and locating the edges of original image.

However, these two kinds of filters will cause the image edge easy to appear multi-pixel width, causing false and redundant edges.

For Prewitt filter, it will cause a low brightness and contrast, and there are some edges missing in the processed image after Prewitt filter. This filter does not have the ability to suppress noise.

Exercise 6

LoG. Laplacian filters are derivative filters used to find edges in images. Since derivative filters are very sensitive to noise, it is common to smooth the image before applying the Laplacian. For example, the image can be smoothed using a Gaussian filter. The two-step process involving Gaussian low-pass filtering followed by Laplacian filtering is called the Laplacian of Gaussian (LoG) operator and will be covered in the first part of the lab.

Implement the LoG operator as a parametric function of the variance and display the results in the boxes.

Lena σ = 1



Peppers σ = 3



Try the effect of LoG filters using different Gaussian widths by changing the variances. What is the general effect after increasing the Gaussian width?

Baboon σ = 1



Baboon σ = 2



Baboon σ = 3



Add your comments here.

When σ increases, the edge line is thicker, and the number of edges is lower. Besides, the contrast and brightness of image decreases in this case. Therefore, there are less detailed information in processed image.

Construct a LoG filter where the mask size is too small for the chosen Gaussian width (*i.e.* the LoG becomes truncated). What is the effect on the output? Define an empirical or analytical rule to determine how large a LoG mask should be in relation to the variance of the underlying Gaussian if severe truncation is to be avoided.

Lena result

σ =3

Kernel size = 19



Peppers result

σ =3

Kernel size = 19



Discussion

When using Gaussian function to process image, the effects are obvious in the interval (-3σ+x, x+3σ). And when σ increases, more neighbourhood pixels can be handled by Gaussian function, and the image will become smoother. Therefore, I choose σ =3.

Then, because the Kernel radius is 3σ, and the anchor (centre) point should be considered, the Kernel size is 6*3+1 = 19.

From the above processed image, after applying this analytical rule, many truncations are avoided, and edges of original images are well maintained.